

Creating a 2 bit adder based on an FPGA

Levi Terry

UAT

RBT131: Digital Logic

Prof. Prater

5/4/25

Intro

We are attempting to create a two-bit adder for an FPGA. The adder takes in 2 2-bit binary numbers and adds them together (hence the name 2-bit adder). Since there are 4 inputs that can be changed, there are $2^4 = 16$ possibilities for the adder. The adder uses both a half adder and a full adder to run its calculations, a0 and b0 run through the half adder, then a1 and b1 along with the carry from the half adder are ran through the full adder to get the final result.

TwoBitBinaryAdder.v

```

1  /*
2      Verilog file for the 2 bit binary adder
3
4      Levi Terry
5      lterry80052@uat.edu
6      RBT131
7      4/17/25
8  */
9
10 // Define the 2BitBinaryAdder module
11 module TwoBitBinaryAdder(a0, a1, b0, b1, s0, s1, c2);
12     // Inputs
13     input a0, a1, b0, b1;
14
15     // Outputs
16     output s0, s1, c2;
17
18     // Create wires
19     wire AND1;
20     wire AND2;
21     wire AND3;
22     wire XOR1;
23
24     // Assignments for half adder 2^0 place value
25     assign s0 = a0 ^ b0;
26     assign AND1 = a0 & b0;
27
28     // Assignments for full adder 2^1 place value
29     assign XOR1 = a1 ^ b1;
30     assign AND2 = a1 & b1;
31     assign s1 = XOR1 ^ AND1;
32     assign AND3 = XOR1 & AND1;
33     assign c2 = AND3 | AND2;
34 endmodule

```

TwoBitBinaryAdder_tb.v

```

1  /*
2  Verilog file for the 2 bit binary adder testbench
3  # of possibilities = 2^4 = 2*2*2*2 = 16
4
5  Levi Terry
6  lterry80052@uat.edu
7  RBT131
8  4/22/25
9  */
10
11 module TwoBitBinaryAdder_tb;
12 // Create wires
13 wire t_s0, t_s1, t_c2;
14
15 // Create registers
16 reg t_a0, t_a1, t_b0, t_b1;
17
18 // How Long the sim should run
19 localparam DURATION = 160;
20
21 // Create TwoBitBinaryAdder
22 TwoBitBinaryAdder my_gate(.a0(t_a0), .a1(t_a1), .b0(t_b0), .b1(t_b1), .s0(t_s0), .s1(t_s1), .c2(t_s2));
23
24 /*
25 b: 1 0 | a: 1 0
26 -----
27 0 0      0 0
28 0 0      0 1
29 0 0      1 0
30 0 0      1 1
31 0 1      0 0
32 0 1      0 1
33 0 1      1 0
34 0 1      1 1
35 1 0      0 0
36 1 0      0 1
37 1 0      1 0
38 1 0      1 1
39 1 1      0 0
40 1 1      0 1
41 1 1      0 0
42 1 1      0 1
43 */
44
45 initial begin
46 // Monitor all wires/registers
47 $monitor(t_a0, t_a1, t_b0, t_b1, t_s0, t_s1, t_s2);
48
49 // Set a to 00
50 t_a0 = 1'b0;
51 t_a1 = 1'b0;
52 // Set b to 00
53 t_b0 = 1'b0;
54 t_b1 = 1'b0;
55
56 // Wait 5 ps
57 #10
58
59 // Set a to 01
60 t_a0 = 1'b1;
61 t_a1 = 1'b0;
62 // Set b to 00
63 t_b0 = 1'b0;
64 t_b1 = 1'b0;
65
66 // Wait 10 ps
67 #10
68

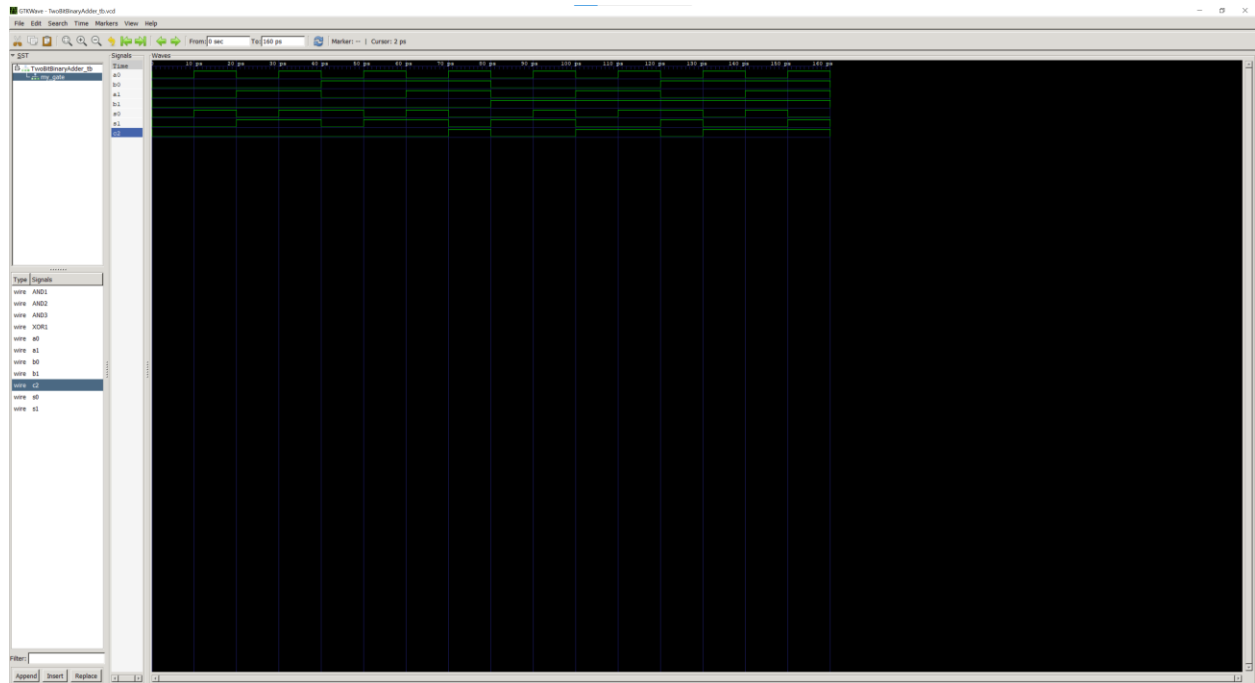
```

```
69 // Set a to 00
70 t_a0 = 1'b0;
71 t_a1 = 1'b1;
72 // Set b to 00
73 t_b0 = 1'b0;
74 t_b1 = 1'b0;
75
76 // Wait 10 ps
77 #10
78
79 // Set a to 00
80 t_a0 = 1'b1;
81 t_a1 = 1'b1;
82 // Set b to 00
83 t_b0 = 1'b0;
84 t_b1 = 1'b0;
85
86 // Wait 10 ps
87 #10
88
89 // Set a to 00
90 t_a0 = 1'b0;
91 t_a1 = 1'b0;
92 // Set b to 00
93 t_b0 = 1'b1;
94 t_b1 = 1'b0;
95
96 // Wait 10 ps
97 #10
98
99 // Set a to 00
100 t_a0 = 1'b1;
101 t_a1 = 1'b0;
102 // Set b to 00
103 t_b0 = 1'b1;
104 t_b1 = 1'b0;
105
106 // Wait 10 ps
107 #10
108
109 // Set a to 00
110 t_a0 = 1'b0;
111 t_a1 = 1'b1;
112 // Set b to 00
113 t_b0 = 1'b1;
114 t_b1 = 1'b0;
115
116 // Wait 10 ps
117 #10
118
119 // Set a to 00
120 t_a0 = 1'b1;
121 t_a1 = 1'b1;
122 // Set b to 00
123 t_b0 = 1'b1;
124 t_b1 = 1'b0;
125
126 // Wait 10 ps
127 #10
128
129 // Set a to 00
130 t_a0 = 1'b0;
131 t_a1 = 1'b0;
132 // Set b to 00
133 t_b0 = 1'b0;
134 t_b1 = 1'b1;
135
136 // Wait 10 ps
137 #10
```

```
138
139 // Set a to 00
140 t_a0 = 1'b1;
141 t_a1 = 1'b0;
142 // Set b to 00
143 t_b0 = 1'b0;
144 t_b1 = 1'b1;
145
146 // Wait 10 ps
147 #10
148
149 // Set a to 00
150 t_a0 = 1'b0;
151 t_a1 = 1'b1;
152 // Set b to 00
153 t_b0 = 1'b0;
154 t_b1 = 1'b1;
155
156 // Wait 10 ps
157 #10
158
159 // Set a to 00
160 t_a0 = 1'b1;
161 t_a1 = 1'b1;
162 // Set b to 00
163 t_b0 = 1'b0;
164 t_b1 = 1'b1;
165
166 // Wait 10 ps
167 #10
168
169 // Set a to 00
170 t_a0 = 1'b0;
171 t_a1 = 1'b0;
172 // Set b to 00
173 t_b0 = 1'b1;
174 t_b1 = 1'b1;
175
176 // Wait 10 ps
177 #10
178
179 // Set a to 00
180 t_a0 = 1'b1;
181 t_a1 = 1'b0;
182 // Set b to 00
183 t_b0 = 1'b1;
184 t_b1 = 1'b1;
185
186 // Wait 10 ps
187 #10
188
189 // Set a to 00
190 t_a0 = 1'b0;
191 t_a1 = 1'b1;
192 // Set b to 00
193 t_b0 = 1'b1;
194 t_b1 = 1'b1;
195
196 // Wait 10 ps
197 #10
198
199 // Set a to 00
200 t_a0 = 1'b1;
201 t_a1 = 1'b1;
202 // Set b to 00
203 t_b0 = 1'b1;
204 t_b1 = 1'b1;
205
206 // Wait 10 ps
```

```
207      #10
208
209      // Set a to 00
210      t_a0 = 1'b0;
211      t_a1 = 1'b0;
212      // Set b to 00
213      t_b0 = 1'b0;
214      t_b1 = 1'b0;
215  end
216
217  initial begin
218      // Creat dumpfile and dump vars
219      $dumpfile("TwoBitBinaryAdder_tb.vcd");
220      $dumpvars(0, TwoBitBinaryAdder_tb);
221
222      // Wait for duration
223      #(DURATION);
224  end
225
226 endmodule
```

GTKWave



Questions

1. How many possibilities do you need for a 2-bit adder?

For a 2-bit adder, there are 16 possibilities since $2^4 = 16$.

2. Show you math work for determining simulation length.

Since there are 16 possibilities and each should run for 10ps, then the simulation should run for 160ps, since $16 \times 10 = 160$